

CAHIER DES CHARGES · V1.0 · PROJET PORTFOLIO

FabriQ.

Assistant d'analyse en langage naturel pour données industrielles. Une question en clair, du SQL gouverné en lecture seule, une réponse transparente avec graphique et explication.

NL → SQL sûr → exécution → graphique → explication · agentique, validé, mesuré

AUTEUR

Mohamed Lassoued

DATE

25 juin 2026

VERSION

1.0 — à réaliser

CIBLE

PME industrielles · Bodenseeraum

00

Sommaire

- 01 Contexte & objectifs
- 02 Positionnement & différenciation
- 03 Périmètre fonctionnel
- 04 Fonctionnalités détaillées
- 05 Exigences non-fonctionnelles
- 06 Architecture technique
- 07 Modèle de données
- 08 Couche de sécurité SQL
- 09 Stratégie d'évaluation
- 10 Stack technique
- 11 Plan de livraison
- 12 Livrables & succès

01

Contexte & objectifs

Pourquoi ce projet existe, et ce qu'il doit prouver.

Dans les PME industrielles, les données opérationnelles — production, stocks, logistique, ventes — restent enfermées dans des bases. Les équipes métier dépendent de l'IT, d'analystes ou de tableaux de bord figés pour répondre à des questions simples. Entre la rigidité des dashboards et la surcharge des analystes, une question comme « *quels produits ont vu leur marge baisser le trimestre dernier ?* » est étonnamment lente à obtenir.

OBJECTIF PRODUIT

Un accès autonome et fiable aux données

Permettre à un utilisateur métier de poser sa question en langage naturel et d'obtenir une réponse juste, sourcée et lisible — sans écrire une ligne de SQL, et sans risque pour la base.

OBJECTIF PORTFOLIO

Démontrer une ingénierie IA senior

Prouver, par un système réel, la maîtrise de l'IA agentique appliquée : pas un wrapper d'API, mais une architecture gouvernée, évaluée et transparente, défendable en entretien.

02

Positionnement & différenciation

Le concept n'est pas neuf. L'exécution, si.

X LA MAUVAISE VERSION

« Pose n'importe quelle question à n'importe quelle base, l'IA répond par magie. » → impression de **junior**, hallucinations, et zéro confiance.

✓ LA BONNE VERSION

Un assistant d'analyse **sûr**, limité à des métriques métier validées en **lecture seule**, avec transparence du SQL, validation, réessais, visualisations et **précision mesurable**.

i

Face aux géants (Power BI Copilot, Databricks Genie...), on ne cherche pas à gagner. La valeur n'est pas « LLM + base » mais « **LLM + contexte métier gouverné + garde-fous déterministes + évaluation** ». Atout différenciant personnel : ce projet rejoue et prolonge l'expérience **Talan** (plateforme IA de migration BI) — autorité totale en entretien.

03

Périmètre fonctionnel

Un MVP étroit et fini bat un produit ambitieux et inachevé.

Inclus — MVP

v1.0

- + Boucle NL → SQL → validation → exécution → réponse
- + Couche de sécurité SQL (lecture seule)
- + Génération automatique de graphiques
- + Questions de clarification
- + Panneau de transparence SQL
- + Harnais d'évaluation
- + Base PostgreSQL industrielle de démonstration
- + ~10 familles de questions

Exclus — volontaire

hors v1

- Multi-tenant / SaaS
- OAuth / authentification avancée
- Facturation cloud
- Recherche vectorielle / RAG documentaire
- Ingestion de PDF
- Streaming temps réel
- Fine-tuning de modèle
- « Décision autonome »

Évolutions — v2

[après](#)

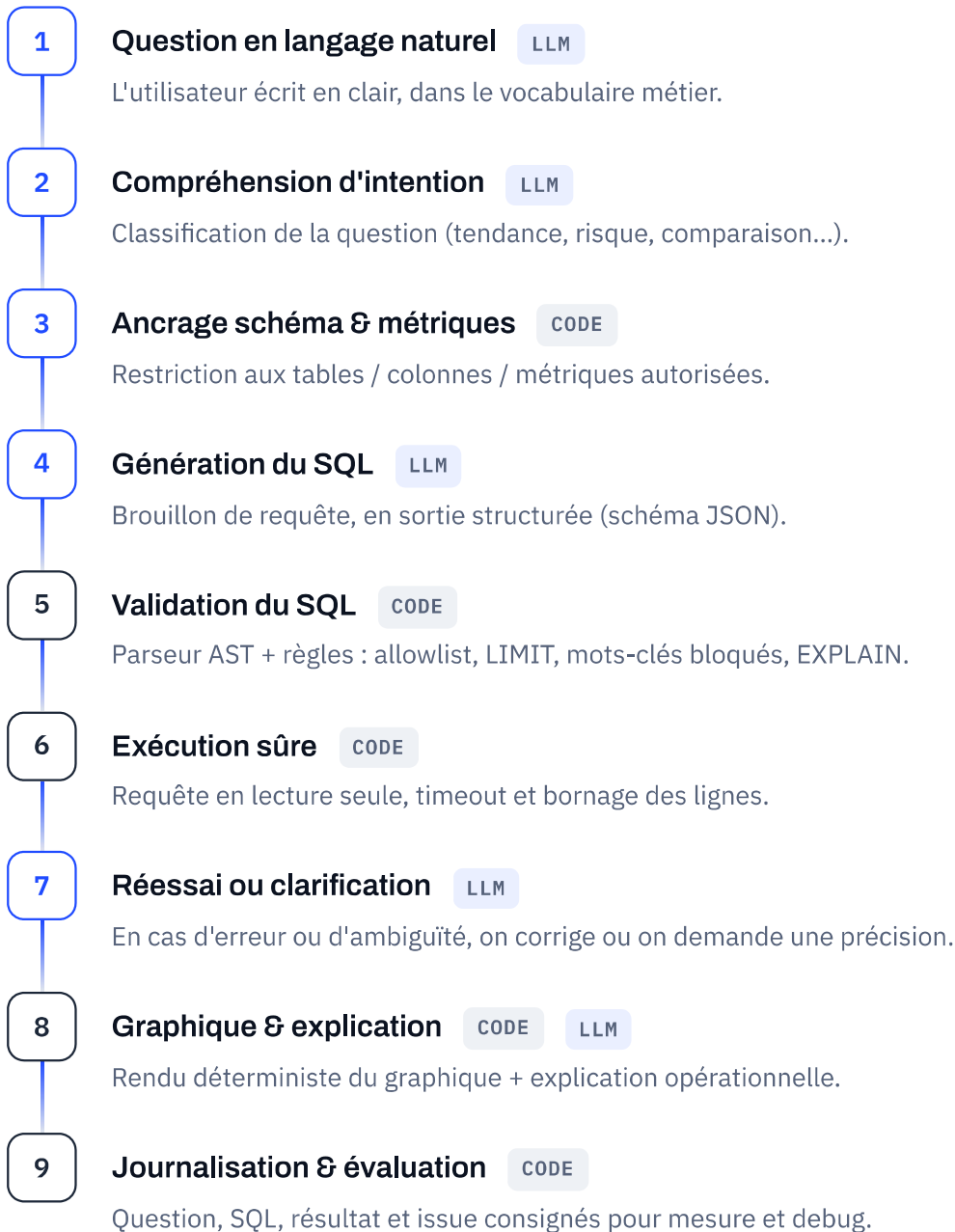
- › Couche sémantique (métriques / synonymes)
- › Détection d'ambiguïté avancée
- › Sélection de graphique déterministe
- › Authentification
- › Déploiement multi-tenant
- › Insights planifiés / alertes

04

Fonctionnalités détaillées

Le flux de bout en bout, et ce que chaque étape garantit.

Flux d'une question



Les 10 familles de questions

FAMILLE	EXEMPLE DE QUESTION
Tendance de marge	« Quels produits ont vu leur marge baisser le trimestre dernier ? »
Risque de rupture	« Quels SKU risquent une rupture dans les 14 prochains jours ? »
Retard fournisseur	« Quels fournisseurs ont été le plus souvent en retard ? »
Efficacité production	« Quelles lignes de production ont eu le plus de défauts ? »
Tendance de CA	« Montre le chiffre d'affaires mensuel par catégorie. »
Vieillessement de stock	« Quels produits sont restés trop longtemps en stock ? »
Coût logistique	« Quelles routes sont devenues plus chères ? »
Retours	« Quels produits ont le taux de retour le plus élevé ? »
Concentration clients	« Quels clients représentent l'essentiel du CA ? »
Détection d'anomalie	« Qu'est-ce qui a changé anormalement le mois dernier ? »

05

Exigences non-fonctionnelles

Les qualités qui font la différence entre un prototype et un système.

SÉCURITÉ

Aucune écriture possible

Le LLM n'a jamais d'accès en écriture. Tout SQL est parsé et validé avant exécution.

FIABILITÉ

Réessais & repli

Réessais bornés, repli déterministe, gestion d'erreurs explicite plutôt que silencieuse.

TRANSPARENCE

SQL visible, pas le scratchpad

On montre le SQL et une explication opérationnelle — jamais le raisonnement interne du modèle.

PERFORMANCE

Exécution bornée

Timeout de requête, LIMIT forcé, coût de requête contrôlé.

OBSERVABILITÉ

Tout est tracé

Journal des questions, du SQL, des résultats et des échecs pour mesurer et corriger.

PORTABILITÉ

Conteneurisé

Exécution via Docker, configuration externalisée (.env), reproductible.

06

Architecture technique

Le LLM propose ; le code déterministe dispose.

PRÉSENTATION

Frontend React

chat · graphiques

Panneau SQL

transparence

ORCHESTRATION

Backend FastAPI

API · sessions

Agent LangGraph

boucle à états

LLM local — Ollama

intention · SQL · texte

DONNÉES & GARDE-FOUS

Couche sécurité SQL

AST · allowlist · limites

PostgreSQL

base de démo (read-only)

Harnais d'évaluation

jeu golden · journal

Principe directeur — le LLM est cantonné à l'interprétation, au brouillon SQL et à la formulation. Les décisions sensibles (permissions, validation, définition des métriques, rendu des graphiques, exécution, évaluation) restent du **code déterministe**. C'est cette séparation qui rend le système « ingénié » plutôt que « prompté ».

07

Modèle de données

Une base de démo industrielle crédible — pas un « jeu jouet ».

TABLE	CONTENU
products	Catalogue produit : référence, catégorie, prix, coût.
orders	Commandes clients : lignes, quantités, montants, dates.
production_batches	Lots de production : ligne, volume, défauts, dates.
inventory_movements	Mouvements de stock : entrées / sorties, quantités, horodatage.
suppliers	Fournisseurs : délais, ponctualité, conditions.
customers	Clients : segment, région, ancienneté.
shipments	Expéditions : route, transporteur, coût, délais.
returns	Retours : produit, motif, quantité, date.
costs	Coûts : par produit / période, pour le calcul de marge.

08

Couche de sécurité SQL

La pièce maîtresse « senior ». La sécurité ne repose jamais sur le prompt seul.

L1

Règles de prompt

Cadrage initial — mais jamais l'unique défense.

L2

Identifiants DB en lecture seule

Le compte utilisé n'a aucun droit d'écriture, point.

L3

Parseur AST

Le SQL est analysé syntaxiquement avant toute exécution.

L4

Allowlist tables & colonnes

Seuls les objets explicitement autorisés sont accessibles.

L5

LIMIT forcé + timeout

Bornage des lignes et du temps d'exécution.

L6

Mots-clés bloqués

INSERT / UPDATE / DELETE / DROP — et pas de multi-statement.

L7

Validation EXPLAIN + sandbox

Plan vérifié, exécution isolée.

À ÉCRIRE NOIR SUR BLANC DANS LE README

« Le LLM n'obtient jamais d'accès en écriture à la base. Le SQL généré est parsé et validé avant d'être exécuté. »

09

Stratégie d'évaluation

La fonctionnalité qui sépare un vrai projet d'un « ça marchait sur mes 3 prompts ».

Chaque question du jeu « golden » est décrite par : intention attendue, tables attendues, motif SQL attendu, résultat attendu, type de graphique attendu, et raison de réussite / échec.



Principe clé — on évalue par **résultat d'exécution**, pas par comparaison de chaîne SQL. Deux requêtes différentes peuvent être sémantiquement équivalentes ; comparer le texte serait une erreur de débutant.

MÉTRIQUE

Validité du SQL

La requête générée est-elle syntaxiquement et sémantiquement valide ?

MÉTRIQUE

Exactitude du résultat

Le résultat exécuté correspond-il au résultat attendu ?

MÉTRIQUE

Correction du graphique

Le type de graphique choisi est-il adapté à la forme des données ?

MÉTRIQUE

Taux d'échec + journal

Combien d'échecs, de quels types — et analyse des cas qui cassent.

10

Stack technique

Tout local, tout gratuit. Couche LLM abstraite et échangeable.

COUCHE	CHOIX	NOTE
Frontend	<code>React</code> + biblio. de graphiques	Panneau de transparence SQL inclus.
Backend	<code>FastAPI</code> (Python)	API, sessions, orchestration.
Agent	<code>LangGraph</code>	Workflow à états, outils, human-in-the-loop.
Base de données	<code>PostgreSQL</code>	Base de démo, compte read-only.
LLM (local, gratuit)	<code>Ollama</code> + <code>Qwen2.5-Coder</code>	Fort en SQL, licence permissive. Alternative dédiée : <code>SQLCoder</code> (Defog). Couche échangeable.
Sorties structurées	Schémas <code>JSON</code> / <code>Pydantic</code>	Pas de « formate joliment » : un contrat, pas des vibes.
Évaluation	Jeu golden + comparateur	Comparaison par résultat d'exécution.
Déploiement	<code>Docker</code> / <code>docker-compose</code>	Config externalisée, reproductible.

11

Plan de livraison

Par jalons. Chaque jalon est démontrable.



JALON 0 — FONDATIONS

Socle technique

Dépôt, docker-compose, base PostgreSQL + données de démo réalistes, couche d'accès en lecture seule.



JALON 1 — BOUCLE CŒUR (MVP)

NL → SQL → réponse

Agent LangGraph complet, couche de sécurité SQL, panneau de transparence, ~10 familles de questions fonctionnelles.



JALON 2 — CONFIANCE

Graphiques + évaluation

Génération automatique de graphiques, harnais d'évaluation, journal d'échecs.



JALON 3 — FINITION PORTFOLIO

Présentation

README pro, schéma d'architecture, GIF de démo (~20 s), démo en ligne optionnelle.



V2 — APRÈS LIVRAISON

Contenu LinkedIn

Couche sémantique, détection d'ambiguïté, sélection de graphique déterministe — chaque ajout devient un post.



Règle d'or — un MVP propre et livré bat un v2 parfait jamais terminé. On vise la fiabilité du cœur avant toute extension.

12

Livrables & critères de succès

Ce qui est rendu, et comment on sait que c'est réussi.

Livrables

- ✓ Dépôt GitHub public, code propre, config externalisée
- ✓ README structuré (problème → archi → sécurité → éval → limites)
- ✓ Schéma d'architecture
- ✓ GIF de démonstration (~20 s)
- ✓ Rapport d'évaluation (jeu golden + précision)
- ✓ Démo en ligne (optionnelle)

Critères de succès

- ✓ La boucle fonctionne de bout en bout sur ≥ 10 familles de questions
- ✓ Aucune écriture possible ; tout SQL passe par la validation
- ✓ Précision mesurée et affichée (comparaison par exécution)
- ✓ README + schéma + GIF présents et soignés
- ✓ Défendable en entretien : archi, tradeoffs, modes d'échec, éval
- ✓ Lien explicite assumé avec l'expérience Talan

Risques & mitigations

RISQUE	MITIGATION
Sur-périmètre (over-scope)	Périmètre serré, MVP d'abord, v2 documentée séparément.
Originalité (le concept existe déjà)	Différenciation par sécurité + évaluation + domaine industriel + transparence ; ancrage Talan.
Fiabilité du text-to-SQL	Garde-fous déterministes, repli, clarification, précision mesurée + analyse d'échecs.
Sur-responsabilisation du LLM	Split strict : LLM = interprétation / brouillon / formulation ; code = permissions, validation, métriques, rendu, exécution, éval.



Cahier des charges v1.0 · Mohamed Lassoued · 25 juin 2026 · Document de spécification — prêt pour la phase de réalisation.